UNIVERSIDAD POLITÉCNICA SALESIANA
CARRERA DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

Sistema de Estimación de Profundidad Estereoscpópica en Tiempo Real con Realidad Aumentada

mediante Microcontroladores ESP32-CAM

Guía de Defensa — Visión Artificial (G8)

Proyecto Integrador Final — Parte I

Autores: Michael Lata · Jordy Romero
Docente: Vladimir Robles Bykbaev
Asignatura: Visión Artificial
Período: Abril – Agosto 2026
Fecha: Junio 2026

DEFENSE GUIDE — INTEGRATING PROJECT (Part I)

Real-Time Stereoscopic Depth Estimation System with Augmented Reality

Criterion	Weight	Main file(s)
1. Capture Architecture & Hardware	20%	`ESP32_CAM_Stereo.ino, CameraSource.cpp`
2. Disparity Map Quality	25%	`StereoProcessor.cpp`
3. Distance Precision & Stability	20%	`KalmanDepth.cpp, StereoProcessor.cpp`
4. Augmented Reality (AR) Effect	15%	`ARRenderer.cpp`
5. Individual Oral Defense	20%	(key questions)
Total	**100%**	

Note: the 85% weighting applies with ESP32-CAM-MB/XIAO-SP cameras; this project uses ESP32-CAM AI-Thinker (**20% weighting**).

1. Criterion 1 — Capture Architecture & Hardware (20%)

What the rubric requires

System built in OpenCV/C++ and communicating with ESP32-CAM microcontrollers whose firmware parameters (AEC, AGC, AWB, XCLK) were deliberately tuned. C++ uses **concurrent threads** for reading both cameras, reducing latency and network jitter.

ESP32-CAM Firmware — `ESP32_CAM_Stereo.ino`

Why AEC / AGC / AWB = OFF

SGBM computes disparity by comparing the intensity of corresponding pixels in the left and right frames. If each camera adjusts its own exposure (AEC) independently, the same physical point produces different pixel values in L and R, introducing systematic errors in the disparity map (ghosting, black holes). With the **same fixed exposure on both cameras**, photometric parity is guaranteed.

Listing 1: Fixed firmware parameters (lines 52–58)

```
constexpr int  CAMERA_XCLK_HZ  = 6000000; // 6 MHz - reduces WiFi jitter
constexpr bool CAMERA_AEC      = false;   // AUTO-EXPOSURE OFF -> manual
constexpr bool CAMERA_AGC      = false;   // AUTO-GAIN     OFF -> manual
constexpr bool CAMERA_AWB      = false;   // AUTO-BALANCE  OFF -> fixed color
constexpr int  CAMERA_EXPOSURE = 400;     // 0-1200; tuned per environment
constexpr int  CAMERA_GAIN     = 0;       // 0-30; minimum = less noise
```

Why XCLK = 6 MHz instead of the default 20 MHz

At higher clock frequencies, the sensor draws more current and generates more heat, increasing jitter in the WiFi transmission. At 6 MHz the ESP32 prioritizes packet integrity over speed, reducing the frame de-synchronization between L and R that would otherwise degrade triangulation.

Parameters can be updated **at runtime without re-flashing** via HTTP: `GET /setexposure?v=400` and `GET /setgain?v=0`.

Concurrent threads — `CameraSource.cpp`

Listing 2: Producer thread for HTTP camera frames

```cpp
// Constructor launches the worker thread immediately
HttpCameraSource::HttpCameraSource(std::string url, int timeoutMs)
    : url_(url), timeoutMs_(timeoutMs) {
    worker_ = std::thread(&HttpCameraSource::workerLoop, this);
}
// Worker continuously downloads JPEG from /capture
void HttpCameraSource::workerLoop() {
    while (running_.load()) {
        // ... libcurl downloads JPEG ...
        {
            std::lock_guard<std::mutex> lk(mutex_);
            latestFrame_ = decoded;   // protected shared write
            hasFrame_    = true;
        }
        framesReceived_.fetch_add(1);
    }
}
// Main loop reads without blocking
bool HttpCameraSource::read(cv::Mat& out) {
    std::lock_guard<std::mutex> lk(mutex_);
    if (!hasFrame_) return false;
    out = latestFrame_.clone();
    return true;
}
```

- Left and right cameras each have their own dedicated producer thread.
- The main (controller) thread consumes the latest frame without waiting.
- `mutex_` protects the shared frame between producer and consumer.
- `std::atomic<bool> running_` stops the thread cleanly without a data race.

2. Criterion 2 — Disparity Map Quality (25%)

What the rubric requires

Dense map, defined edges, low noise. Justified application of preprocessing (CLAHE) and postprocessing (WLS) filters coupled to SGBM.

Full pipeline — `StereoProcessor.cpp`

Raw frame $\xrightarrow{\text{remap}}$ Rectified $\xrightarrow{\text{cvtColor}}$ Grayscale $\xrightarrow{\text{CLAHE}}$ Enhanced contrast $\xrightarrow{\text{SGBM}}$ Q16.4 disparity $\xrightarrow{\text{WLS}}$ Filtered d

Listing 3: Step 1: Rectification (lens distortion correction)

```cpp
// remap applies the pre-computed calibration maps to remove lens
// distortion and align epipolar lines horizontally
cv::remap(rawLeft,  result.leftRect,  calib_->map1x, calib_->map1y, INTER_LINEAR);
cv::remap(rawRight, result.rightRect, calib_->map2x, calib_->map2y, INTER_LINEAR);
```

Listing 4: Step 3: CLAHE preprocessing (lines 88–92)

```cpp
// clipLimit=2.0: caps the maximum amplification of the local histogram
// tileSize 8x8: divides the image into tiles for local equalization
clahe_ = createCLAHE(cfg.claheClipLimit, Size(8, 8));
clahe_->apply(grayL, grayL);
clahe_->apply(grayR, grayR);
```

Why CLAHE before SGBM

SGBM finds correspondences by comparing block intensity. In areas with uneven lighting or shadows, local contrast is low and the left block does not look like its right counterpart. CLAHE equalizes the histogram **locally** without saturating bright areas, improving discrimination at edges and fine textures.

Listing 5: Step 4: SGBM – key parameters (lines 29–35)

```
sgbm_ = StereoSGBM::create(cfg.minDisp, numDisp, blockSz);
// P1 and P2 control the smoothness of the disparity map.
// The ratio P2 = 4*P1 follows Hirschmuller's recommendation.
sgbm_->setP1(8  * 3 * blockSz * blockSz);
sgbm_->setP2(32 * 3 * blockSz * blockSz);
sgbm_->setUniquenessRatio(cfg.uniquenessRatio);
sgbm_->setMode(StereoSGBM::MODE_SGBM_3WAY);
```

Listing 6: Step 5: WLS postprocessing (lines 101–108)

```
// Right matcher computes R->L disparity required by WLS
rightMatcher_->compute(grayR, grayL, disp16R);
Mat filtered;
// WLS fills holes and smooths edges (guided filter principle)
wlsFilter_->filter(disp16L, result.leftRect, filtered, disp16R);
filtered.convertTo(result.disparity, CV_32F, 1.0 / 16.0);
```

SGBM vs BM — why we use SGBM

BM (Block Matching) searches only horizontally. SGBM (Semi-Global Block Matching) adds smoothness constraints in 8 directions (semi-global), producing denser maps with fewer holes, especially in low-texture regions.

Listing 7: Step 6: Invalid-zone fill (horizontal propagation)

```
// SGBM leaves ~numDisp black columns on the left with no valid data.
// The first valid value in each row is propagated leftward.
for (int y = 0; y < result.disparity.rows; ++y) {
    float* row = result.disparity.ptr<float>(y);
    float fill = 0.0f;
    for (int x = 0; x < result.disparity.cols; ++x) {
        if (row[x] > 0.5f) { fill = row[x]; break; }
    }
    if (fill <= 0.0f) continue;
    for (int x = 0; x < result.disparity.cols; ++x) {
        if (row[x] > 0.5f) break;
        row[x] = fill;
    }
}
```

Modification note: to fill **vertically** (column by column) instead, make the outer loop iterate over `x` and the inner over `y`, switching from `ptr<float>(y)[x]` (fast row-major access) to `at<float>(y,x)` (random access).

3. Criterion 3 — Distance Precision & Stability (20%)

What the rubric requires

Correct reprojection of the 3D matrix, Z coordinate in real units (cm/mm), stable reading with a 1D Kalman Filter, fluctuation $\leq \pm 3$ cm in a static scene.

3D Reprojection

Listing 8: Step 7: reprojectImageTo3D (line 131)

```
// Q is the reprojection matrix computed during stereo calibration.
// It converts the disparity map into a 3D point cloud (X, Y, Z in meters).
reprojectImageTo3D(result.disparity, result.points3D, calib_->Q, true);
```

Listing 9: Robust measurement: median over a 10x10 px region

```
float StereoProcessor::medianDepthAt(const Mat& points3D, int cx, int cy, int r) {
    // Collect all valid Z values in a square of radius r
    for (int y = y0; y <= y1; ++y)
        for (int x = x0; x <= x1; ++x) {
            const float z = points3D.at<Vec3f>(y, x)[2];
            // Keep only finite values in the plausible range (5 cm - 10 m)
            if (std::isfinite(z) && z > 0.05f && z < 10.0f)
                zVals.push_back(z);
        }
    // nth_element: O(n) - equivalent to median without sorting the entire vector
    std::nth_element(zVals.begin(), zVals.begin() + mid, zVals.end());
    return zVals[mid];
}
```

1D Kalman Filter — `KalmanDepth.cpp`

Listing 10: Scalar Kalman filter (the 3 key equations)

```
double KalmanDepth::update(double measurement) {
    // PREDICTION: uncertainty grows over time (process noise q)
    covariance_ += q_;

    // KALMAN GAIN: balances trust in prediction vs. new measurement
    const double gain = covariance_ / (covariance_ + r_);

    // UPDATE: corrects the estimate with the new measurement
    estimate_   += gain * (measurement - estimate_);
    covariance_  = (1.0 - gain) * covariance_;
    return estimate_;
}
```

Variable	Meaning	Effect if increased
`q_`	Process noise	Kalman trusts measurement more (fast response)
`r_`	Measurement noise	Kalman trusts prediction more (smoother output)
`gain`	Kalman gain	0 = ignore measurement; 1 = ignore prediction

Key **C** resets the central Kalman filter (`kalmanCenter_.reset()`). Key **R** starts/stops CSV logging with columns: `frame, elapsed_ms, z_raw_cm, z_kalman_cm, fps`.

Validation at 30 / 60 / 90 cm

`python/collect_measurements.py` automates data collection: 50 samples per distance, computes mean, standard deviation, and absolute error, and verifies that `std` ≤ 3.0 cm (rubric requirement).

4. Criterion 4 — Augmented Reality Effect (15%)

What the rubric requires

An original computational visual effect overlaid on the rectified color transmission. The effect must react **dynamically** to the measured distance (scale, position, or behavior). Exclusive design unique to the group.

Effect description — `ARRenderer.cpp`

The **Energy Ball** effect has three stacked layers, all modulated by Z:

Layer 0: Depth-driven pixel distortion field (`applyDepthField`)

Listing 11: Depth-reactive wave distortion

```
// Strength inversely proportional to Z^3: drops off very fast with distance
const float z3       = zFocusMeters * zFocusMeters * zFocusMeters;
const float strength = std::clamp(0.6f / std::max(z3, 1e-4f), 0.0f, 70.0f);
const float speed    = 10.0f / std::max(0.15f, zFocusMeters);

// Sinusoidal pixel displacement for foreground pixels only
const float ox = strength * std::sin(y * 0.10f + t * speed);
const float oy = strength * std::cos(x * 0.10f + t * speed * 0.75f);
dst[x] = src.at<cv::Vec3b>(sy, sx);  // reads displaced pixel
```

Dynamic behavior: at 20 cm the wave displaces up to 70 px at high speed. At 80 cm the perturbation drops to 2 px. Beyond 80 cm there is no effect (`strength < 2.0f`).

Layer 1: Radial color tint (`applyPixelVignette`)

Listing 12: Tint color reactive to distance

```
// Tint color changes with distance (in BGR):
if (depthCm < 40.0f)       { tB=20;  tG=60;  tR=255; } // RED   (danger: very close)
else if (depthCm < 80.0f)  { tB=20;  tG=220; tR=80;  } // GREEN (optimal zone)
else                        { tB=240; tG=160; tR=20;  } // CYAN  (far)
```

Layer 2: Pulsing aura with orbital sparks (`drawEnergyAura`)

Listing 13: Pulse and spark speed reactive to Z

```
// Soft 3 Hz pulse -- radius oscillates slightly
const float pulse  = 0.90f + 0.10f * std::sin(phase * 3.0);
// Closer -> more sparks and faster orbit
const float proximity  = std::clamp(20.0f / depthCm, 0.15f, 1.0f);
const int   sparkCount = 6 + int(8.0f * proximity);   // 6 to 14 sparks
const double spinSpeed = 1.0 + 2.0 * proximity;        // up to 3x faster
```

Distance	Color	Behavior
< 40 cm (close)	Red	Strong wave, 14 fast sparks, intense tint
40–80 cm (optimal)	Green	Moderate wave, 10 sparks, soft tint
> 80 cm (far)	Cyan	Minimal wave, 6 slow sparks

5. Criterion 5 — Individual Oral Defense (20%)

Likely questions and key answers

1. What is matrix Q and what does `reprojectImageTo3D` do?

Q is the 4×4 reprojection matrix computed by `cv::stereoRectify`. It transforms disparity d into 3D coordinates:

$$\begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix} = Q \cdot \begin{bmatrix} u \\ v \\ d \\ 1 \end{bmatrix} \quad \Rightarrow \quad Z = \frac{f \cdot B}{d}$$

where f is the focal length and B is the baseline (distance between cameras). `reprojectImageTo3D` applies this operation to every pixel of the disparity map, producing the 3D point cloud stored in `result.points3D`.

2. How do threads work in the system?

Each `HttpCameraSource` launches a `std::thread` (producer thread) that continuously downloads JPEG frames via libcurl. The main thread (consumer) calls `read()`, which grabs the latest frame under a `mutex`. This producer-consumer architecture ensures that HTTP downloads do not block SGBM processing.

3. How are SGBM parameters tuned?

- `numDisparities`: search range in pixels (must be a multiple of 16). Larger = sees farther but slower.
- `blockSize`: comparison block size (odd, ≥ 3). Larger = smoother but loses detail.
- `P1, P2`: smoothness penalties. $P2 = 4 \cdot P1$ follows Hirschmuller's guideline. High $P2$ = smooth plane transitions.
- `uniquenessRatio`: rejects a match if the second-best candidate is within X% of the best.
- `speckleWindowSize / Range`: removes isolated blobs (salt-and-pepper noise).

All parameters are adjustable **in real time** via the trackbar control window — no restart needed.

4. What does key C do?

It resets the central Kalman filter (`kalmanCenter_.reset()`), setting `initialized_ = false`. The next measurement is accepted directly as the new initial estimate, discarding all previous history. Useful when the target object changes position abruptly.

5. Why use the median instead of the mean to measure Z?

The median is robust to outliers. Pixels at object edges have erroneous disparities due to occlusion. The mean would be contaminated by those values; the median ignores them as long as more than 50% of pixels in the region are valid.

6. What is the difference between Z raw and Z Kalman in the HUD?

`Z raw` is the direct output of `medianDepthAt` (may vary frame to frame). `Z Kalman` is the filtered estimate: more stable, with a slight lag, but meets the $\leq \pm 3$ cm fluctuation requirement of the rubric.

7. Why does the AR effect apply only to the foreground?

In `applyDepthField` there is a threshold: `const float fgLimit = zFocusMeters + 0.50f`. Only pixels with $Z <$ focus $+ 0.5$ m receive the distortion. Background pixels (wall, floor) are untouched, giving the visual impression that the effect wraps only around the measured object.

8. If asked to change Step 6 fill direction from horizontal to vertical, what changes?

The outer loop changes from `for y` to `for x`, and the inner from `for x` to `for y`. Access changes from `ptr<float>(y)[x]` (efficient row-major) to `at<float>(y,x)` (random access, required when `x` is the outer axis).

File-to-criterion reference

File	Responsibility	Criterion
`ESP32_CAM_Stereo.ino`	Firmware: AEC/AGC/AWB, XCLK, HTTP endpoints	1
`CameraSource.cpp/.hpp`	Producer thread, mutex, non-blocking read	1
`StereoCalibration.cpp`	Loads `stereo_calib.yml`, rectification maps, Q	2, 3
`StereoProcessor.cpp`	remap → CLAHE → SGBM → WLS → reproject	2, 3
`KalmanDepth.cpp`	Scalar 1D Kalman filter	3
`ARRenderer.cpp`	Distortion field, radial tint, pulsing aura	4
`DisplayManager.cpp`	HUD, trackbars, windows	1, 2, 3
`StereoController.cpp`	Main MVC loop, CSV logger, key handling	all
`collect_measurements.py`	30/60/90 cm validation, CSV results	3
`find_cameras.py`	Network scan to locate ESP32-CAMs	1